

Пояснительные материалы к проекту

1. Описание проекта

Название: «Code Indexer» — API для навигации по кодовой базе

Цель: Создать REST-сервис на Python, который индексирует Python-файлы и позволяет искать функции и классы по имени или описанию.

Решение проблемы: В крупных IT-проектах новичкам сложно ориентироваться в кодовой базе. Сервис структурирует код и отвечает на запросы через API.

2. Схема базы данных (PostgreSQL)

Таблица `files`:

Колонка	Тип	Описание
<code>id</code>	INTEGER PRIMARY KEY	Уникальный идентификатор файла
<code>name</code>	TEXT NOT NULL UNIQUE	Имя файла
<code>path</code>	TEXT NOT NULL	Полный путь к файлу

Таблица `code_entities`:

Колонка	Тип	Описание
<code>id</code>	INTEGER PRIMARY KEY	Уникальный идентификатор
<code>file_id</code>	INTEGER FOREIGN KEY	Ссылка на <code>files(id)</code>
<code>entity_type</code>	TEXT NOT NULL	'function' или 'class'
<code>name</code>	TEXT NOT NULL	Имя функции/класса
<code>start_line</code>	INTEGER NOT NULL	Номер начальной строки
<code>end_line</code>	INTEGER NOT NULL	Номер конечной строки
<code>docstring</code>	TEXT	Документация (описание)

Индексы:

```
-- Индекс делает колонку name отсортированной для бинарного поиска
CREATE INDEX IF NOT EXISTS idx_entity_name ON code_entities(name);

-- Индекс сортирует имена в нижнем регистре (LOWER(name)) для
регистронезависимого бинарного поиска
CREATE INDEX IF NOT EXISTS idx_entity_name_lower ON code_entities(LOWER(name));

-- Составной индекс сортирует сначала по типу (entity_type), потом по имени
(name) для бинарного поиска по фильтру "тип + имя"
CREATE INDEX IF NOT EXISTS idx_entity_type_name ON code_entities(entity_type,
name);

-- Индекс сортирует file_id для быстрого поиска всех функций и классов
CREATE INDEX IF NOT EXISTS idx_entity_file_id ON code_entities(file_id);
```

Почему такие индексы?

- `idx_entity_name` — для быстрого поиска по ключевому слову
- `idx_entity_type_name` — для фильтрации по типу (class/function)
- Без индексов поиск на больших объемах данных (10000+ записей) будет медленным.

Почему я использовал PostgreSQL?

Я выбрал PostgreSQL, так как он лучше подходит для production-среды: поддерживает параллельные запросы, безопасность и масштабирование. SQLite удобен для прототипов, но не для реального API.

3. Архитектура проекта

```
Архив кода/
├── indexer.py      # Индексатор (парсит .py файлы)
├── database.py     # Работа с БД (PostgreSQL)
├── main.py         # FastAPI сервер
├── config.py       # Конфигурация
├── generate_test_data.py # Генерация тестовых данных
├── requirements.txt # Зависимости
├── Dockerfile      # Контейнеризация (бонус)
├── docker-compose.yml # Оркестрация (бонус)
└── README.md       # Документация
```

Технологический стек:

- Python 3.11+
- `ast` — встроенный парсер Python кода
- FastAPI — REST API фреймворк
- PostgreSQL — база данных
- Docker — контейнеризация (бонус)

4. API Эндпоинты

Метод	Эндпоинт	Описание	Возвращает
GET	/api/files	Список всех файлов с количеством сущностей	[{"name": "...", "entities_count": 5}]
GET	/api/files/{name}/structure	Структура файла (функции и классы)	[{"type": "...", "name": "...", "start_line": 1, ...}]
GET	/api/search?q={keyword}	Поиск по имени или docstring	[{"file": "...", "type": "...", "name": "...", ...}]

Бонусные эндпоинты:

- GET /api/search?q={keyword}&type=class — фильтрация по типу
- GET /app/stats — сводная статистика
- Пагинация через ?limit=10&offset=0

Важно: При пустых результатах возвращается [], а не ошибка 500.

5. Индексатор (Часть А)

Принцип работы:

1. Обходит все .py файлы в директории
2. Использует модуль ast для парсинга (без сторонних библиотек)
3. Извлекает:
 - Имена классов и функций
 - Строки начала и конца
 - Docstring (описание)
4. Сохраняет в базу данных

6. Бонусные возможности (реализованы)

- Фильтрация поиска по типу (?type=class/function)
- Эндпоинт /app/stats со статистикой
- Пагинация (?limit=&offset=)
- Docker-контейнеризация
- PostgreSQL вместо SQLite

7. Выводы

Разработанный сервис «»:

- Полностью соответствует требованиям задания(**PostgreSQL вместо SQLite**)
- Индексирует Python файлы с помощью встроенного модуля ast
- Предоставляет REST API с тремя основными эндпоинтами
- Реализованы все бонусные функции
- Проект контейнеризирован и запускается одной командой

Ссылка на GitHub: <https://github.com/NikMat2007/code-indexer>